Chapter 6: Concurrency Control Techniques

Thursday, December 25, 2025 8:05 PM

Q1] Need for Concurrency Control & Locking
"Why is concurrency control required in DBMS? Explain the problems caused by concurrent execution of transactions and describe locking techniques (shared and exclusive locks) used to ensure database consistency."
Covers:
→ **Why concurrency control is needed**
→ **Lost update, dirty read (just idea level)**
→ **Locking concept**
→ **Shared & Exclusive locks**
Ans:

- Why Concurrent Control is Required:
  - In a DBMS:
    - Multiple transactions run at the same time.
    - They may access the same data item.
  - Without control, concurrent execution can:
    - Produce incorrect results
    - Violate database consistency
  - Concurrency control ensures that concurrent transactions behave as if they were executed serially.
- Problems Due to Concurrent Execution:
  - **Lost Update:**
    - Two transactions update the same data
    - One update overwrites the other
    - Result: One update is lost
  - **Dirty Read:**
    - A transaction reads data written by another uncommitted transaction
    - Result: Incorrect or temporary data is read
  - **Unrepeatable Read:**
    - A transaction reads the same data twice
    - Value changes due to another transaction
    - Result: Inconsistent read
  - **Incorrect Summary:**
    - One transaction computes aggregate values
    - Another transaction updates data during computation
    - Result: Wrong calculation
- Locking Technique for Concurrency Control:
  - Locking is the most common method used to control concurrency.
  - A transaction must acquire a lock on a data item before accessing it.
- Types of Locks:
  - **Shared Lock (S):**
    - Used to read operation
    - Multiple transaction can hold S-lock simultaneously
    - Example:
      - ```
        T1: Read(A)
        T2: Read(A)
        ```
      - Allowed
  - **Exclusive Lock (X):**
    - Used for write operation
    - Only one transaction can hold X-lock
    - Example:
        ```

```
T1: Write(A)
T2: Read(A)
```

- Not allowed

- How Locking Ensures Consistency:
  - Prevents simultaneous conflicting operations
  - Ensures isolation between transactions
  - Maintains serializable execution
- One-Line Exam Summary:
  - Concurrency control is required to ensure correct execution of concurrent transactions, and locking techniques using shared and exclusive locks prevent inconsistencies such as lost updates and dirty reads.

Q2] Two-Phase Locking (2PL)
"Explain the Two-Phase Locking (2PL) protocol. Describe its phases and explain how 2PL ensures serializability in concurrent transaction execution."
Covers:
→ **What is 2PL**
→ **Growing & shrinking phase**
→ **Why serializable**
→ **Core exam concept**
Ans:

- What is Two-Phase Locking (2PL)?:
  - Two-Phase Locking (2PL) is a locking protocol in which:
    A transaction locks data items in one phase and releases them in another phase.
  - This rule ensures serializable execution of transactions.
- Phases of 2PL:
  - **Growing Phase:**
    - Transaction can acquire locks
    - Transaction cannot release any lock
  - **Shrinking Phase:**
    - Transaction releases locks
    - Transaction cannot acquire any new lock
    Once a lock is released, the transaction cannot request another lock.
- Why 2PL Ensures Serializability:
  - All conflicting operations are ordered
  - Locking prevents interleaving of conflicting actions
  - Resulting schedule is conflict-serializable
    Every schedule generated by 2PL is serializable.
- Key Point for Exams:
  - 2PL ensures serializability
  - But does not prevent deadlocks
- One-Line Exam Summary:
  - Two-Phase Locking (2PL) is a concurrency control protocol in which each transaction has a growing phase for acquiring locks and a shrinking phase for releasing locks, guaranteeing conflict serializability.

Q3] Deadlock and Starvation
"What is deadlock in DBMS? Explain how deadlock occur in locking-based concurrency control systems. Also explain starvation and distinguish it from deadlock."
Covers:
→ **Deadlock meaning**
→ **Cause of deadlock**
→ **Starvation**

→ **Deadlock vs starvation**

Ans:

- Deadlock in DBMS:
  - Deadlock occurs when:
    Two or more transactions wait indefinitely for each other to release locks.
  - Simple Example:
    - T1 holds lock on A, requests lock on B
    - T2 holds lock on B, requests lock on A
      Both wait forever --> Deadlock
- Causes of Deadlock:
  - Exclusive locks
  - Circular wait condition
  - Improper lock ordering
  - Two-Phase Locking (2PL)
- Starvation:
  - Starvation occurs when:
    A transaction waits indefinitely because other transactions keep getting priority.
  - Example:
    - Long transaction repeatedly delayed
    - Short transaction continuously executed
- Deadlock vs Starvation:
  -

Aspect	**Deadlock**	**Starvation**
Waiting	Circular	Indefinite
Cause	Mutual lock dependencies	Priority or scheduling
Resolution	Detection/Prevention	Fair scheduling
State	System stuck	System running

- One-Line Exam Summary:
  - Deadlock is a situation where transactions wait indefinitely due to circular lock dependencies, whereas starvation occurs when a transaction is repeatedly denied access due to priority policies.

Q4] Deadlock Handling Techniques

"Explain the techniques used to handle deadlocks in DBMS, including deadlock prevention, detection, and recovery."

Covers:

→ **Prevention**

→ **Detection**

→ **Recovery**

→ **Just core ideas**

Ans:

- Deadlock Prevention:
  - Idea: Ensure deadlock conditions never occur.
  - Techniques:
    - **Resource ordering** -- request locks in a fixed order
    - **No hold and wait** -- acquire all locks at once
    - **Preemption** -- force release of locks
  - Reduces concurrency
- Deadlock Detection:
  - Idea: Allow detection, then detect it.
  - Technique:
    - **Wait-for graph**

        - Cycle in graph --> deadlocks exists
    - Better concurrency
    - Detection Overhead
- Deadlock Recovery:
    - After detection:
        - Abort one or more transactions
        - Release their locks
        - Restart aborted transactions
- Key Exam Point:
    - Prevention avoids deadlocks
    - Detection + recovery handles deadlocks after occurrence
- One-Line Exam Summary:
    - Deadlocks in DBMS are handled using prevention techniques to avoid circular waits or by detection using wait-for graphs followed by recovery through transaction rollback.

Q5] Timestamp-Based Concurrency Control
"Explain timestamp ordering technique for concurrency control in DBMS. Describe how read and write operations are ordered using timestamps and compare it with lock-based protocols."
Covers:
→ **Timestamp ordering**
→ **Read & write rules**
→ **Difference vs locking**
Ans:

- Timestamp Ordering Technique:
    - In timestamp-based concurrency control:
        - Each transaction is assigned a unique timestamp
        - Transactions are executed in timestamp order
        - Older transaction --> higher priority
    - No locks are used
- Timestamp Used:
    - For each data item X:
        - RTS(X) --> Read Timestamp
        - WTS(X) --> Write Timestamp
- Read Rule:
    - Transaction Ti can read X if:
        - `TS(Ti) ≥ WTS(X)`
        - Otherwise --> Ti is aborted
- Write Rule:
    - Transaction Ti can write X if:
        - `TS(Ti) ≥ RTS(X) AND TS(Ti) ≥ WTS(X)`
        - Otherwise --> Ti is aborted
- Advantages:
    - No deadlocks
    - High concurrency
    - Simple ordering logic
- Disadvantages:
    - Frequent transaction rollbacks
    - Starvation possible
    - Overhead of timestamp management
- Timestamp vs Lock-Based Protocols:

Aspect	Timestamp Ordering	Lock-Based
Locks	No	Yes
Deadlock	No	Yes
Rollback	Frequent	Less
Complexity	Medium	Low

- One-Line Exam Summary:
  - Timestamp-based concurrency control orders transactions using timestamps instead of locks, ensuring serializability without deadlocks but with possible transaction rollbacks.